

Optimization V

Some Optimization Algorithms

Quang-Thanh Tran

National Economics University
Department of Mathematical Economics

June 12, 2025

1 Linear Programming

- Integer Linear Programming (discrete optim)
- Shortest Path (Dijkstra)
- Maximum Flow (Ford–Fulkerson)

2 Root Finding

- Bisection
- Newton
- Broyden

3 Convex Optimization [incomplete / optional]

- Golden Section Search
- Newton
- Gradient Descent
- Nesterov Accelerated Descent

• Code for the algorithms: [link](#)

• To run the .py files, paste the script content into [here](#) and press **Run**.
Remember to adjust the content for each specific problem.

Linear Programming

Integer Linear Programming problem.

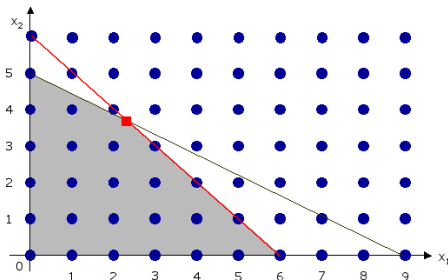


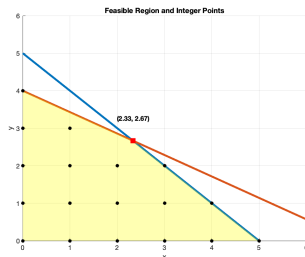
Figure: The LP feasible region is the gray area. But ILP only consists of the blue integer dots lying within the gray region.

Today we learn the Branch-and-Bound algorithm. Other methods include:

- Cutting Plane
- Enumerative methods
- Dynamic Programming (commonly used for knapsack)
- Heuristic (approximate): Local search, Greedy, Tabu, GA, etc.

$$\begin{aligned} \max z &= 5x_1 + 6x_2 \\ \text{s.t. } x_1 + x_2 &\leq 5 \\ 4x_1 + 7x_2 &\leq 28 \\ x_1, x_2 &\geq 0, x_1, x_2 \in \mathbb{Z} \end{aligned}$$

By simple geometric methods, the optimal solution obtained is $(7/3, 8/3)$ with $z = 27.67$. However, these are not integers. The optimal integer solution must be $(3, 2)$ with $z = 27$.

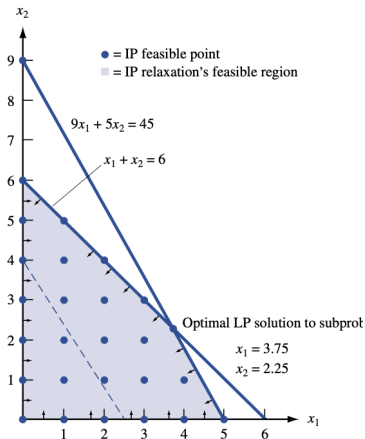


Algorithm description

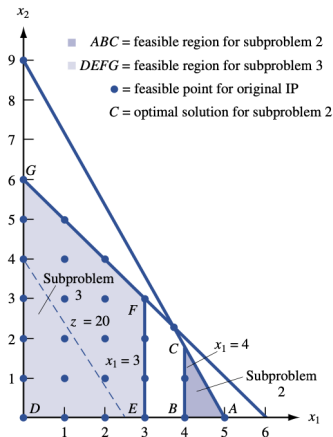
- Step 1: Solve the LP problem after relaxing the integer constraint (LP relaxation). Store the result as LB.
- Step 2: If the solution is integer, stop.
- If not, BRANCH: Choose a variable with the largest fractional part, split the problem into 2 smaller subproblems.
- Step 3: Bound. For each subproblem, do the following:
 - 1 Solve the small LP relaxation problem.
 - 2 If the solution is infeasible, prune this branch.
 - 3 If the solution is integer and better than the best known integer solution, update LB.
 - 4 If the solution gives a better result, go back to Step 2.
- Step 4: Choose the solution with the highest LB.

$$\begin{aligned}\max z &= 8x_1 + 5x_2, \\ \text{s.t. } x_1 + x_2 &\leq 6 \\ 9x_1 + 5x_2 &\leq 45 \\ x_1, x_2 &\geq 0; x_1, x_2 \in \mathbb{Z}\end{aligned}$$

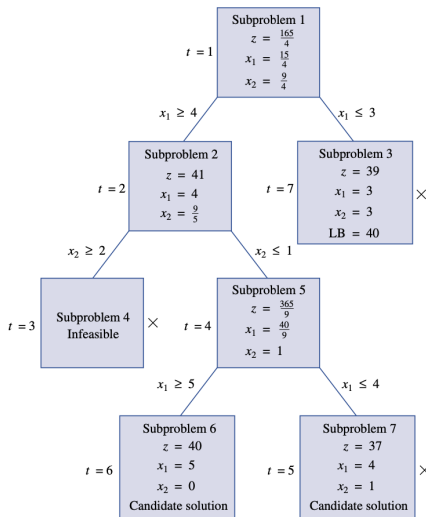
- Find the LP relaxation solution of the original problem. Answer: $(15/4, 9/4)$.
- Using BnB, prove the best solution is $(5, 0)$.



(a) Problem 1: LP relaxation



(b) Problem after branching



Problem 1

$$\begin{aligned} \max z &= 4x_1 + 3x_2 \\ \text{s.t. } 4x_1 + 9x_2 &\leq 26 \\ 8x_1 + 5x_2 &\leq 17 \\ x_1, x_2 &\geq 0; x_1, x_2 \in \mathbb{Z} \end{aligned}$$

Answer: (2, 0), $f = 8$, depth = 6.

Problem 3

$$\begin{aligned} \max z &= 3x_1 + x_2 \\ \text{s.t. } 5x_1 + 2x_2 &\leq 10 \\ 4x_1 + x_2 &\leq 7 \\ x_1, x_2 &\geq 0; x_2 \in \mathbb{Z} \end{aligned}$$

Answer: (2, 2), $f = 18$, depth = 2.

Problem 2

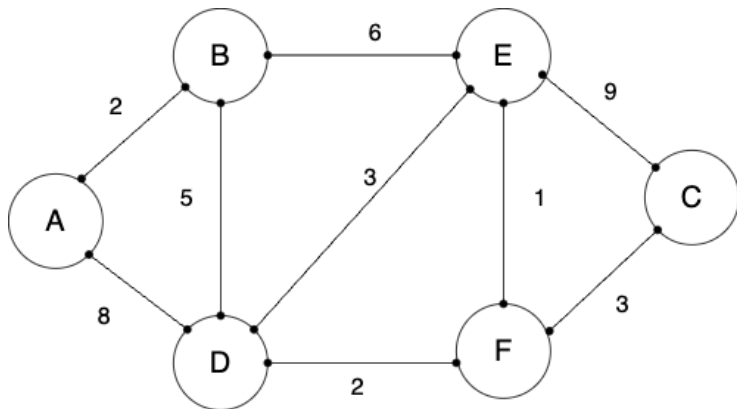
$$\begin{aligned} \max z &= 4x_1 + 5x_2 \\ \text{s.t. } 3x_1 + 2x_2 &\leq 10 \\ x_1 + 4x_2 &\leq 11, 3x_1 + 3x_2 \leq 13 \\ x_1, x_2 &\geq 0; x_1, x_2 \in \mathbb{Z} \end{aligned}$$

Answer: (1.2, 2), $f = 5.6$, depth = 2.

Problem 4

$$\begin{aligned} \max z &= 4x_1 + 3x_2 + x_3 \\ \text{s.t. } 3x_1 + 2x_2 + x_3 &\leq 7 \\ 2x_1 + x_2 + 2x_3 &\leq 11 \\ x_1, x_2, x_3 &\geq 0; x_2, x_3 \in \mathbb{Z} \end{aligned}$$

Answer: (1/3, 3, 0), $f = 10.33$, d = 2.



Today we learn the Dijkstra algorithm. Other methods include:

- Bellman-Ford if there are negative weights
- Floyd-Warshall
- Heuristic: A-star, BFS (breadth-first search)

Travel from A to C. Distance from source $s \in A$.

1 Initialization:

- distance to source A: $\text{dist}(s) = 0$
- distance to all other nodes $\text{dist}(v) = \infty$ for all $v \neq s$.
- mark all nodes as unvisited.

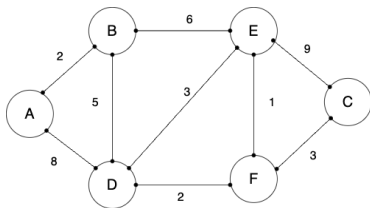
2 Main loop

- Start with an unvisited node a (for the first iteration, $a \leftarrow s$)
- For each neighbor v of a :
 - If v is unvisited, compute: $\text{alt} = \text{dist}(a) + \text{weigh}(a, v)$
 - if $\text{alt} < \text{dist}(v)$, update $\text{dist}(v) = \text{alt}$. Store previous node as a .
- Mark a as visited (no revisit)
- Choose a new current node from unvisited nodes with minimal distance, set that node to a and go back to the start of the main loop.

3 End condition: When all nodes are visited. Then reconstruct the path backward.

Example

Use the algorithm to find the shortest path from A to C.



- Optimal path: A-B-D-F-C.
- Number of iterations: 5.

Use the following table to update

Node	Shortest path	Previous node
⋮	⋮	⋮

List:

- Visited node: $\{A\}$
- Unvisited node: $\{B, C, D, E, F\}$

Node	Shortest path	Previous node
A	0	
B	2	A
D	8	A
E	∞	
F	∞	
C	∞	

Currently visiting: A

Visited:

Unvisited: B, D, E, F, C

Node	Shortest path	Previous node
A	0	
B	2	A
D	$2+5=7$; 8	B (updated)
E	8	B
F	∞	
C	∞	

Currently visiting: B

Visited: A

Unvisited: D, E, F, C

Node	Shortest path	Previous node
A	0	
B	2	A
D	7	B
E	8 ; $7+3=10$	B (not updated)
F	$7+2=9$	D
C	∞	

Currently visiting: D

Visited: A, B

Unvisited: E, F, C

Node	Shortest path	Previous node
A	0	
B	2	A
D	7	B
E	8	B
F	$9 = 8+1$	D (not updated)
C	$8+9=17$	E

Currently visiting: E

Visited: A, B, D

Unvisited: F, C

Node	Shortest path	Previous node
A	0	
B	2	A
D	7	B
E	8	B
F	9	D
C	17 ; $9+3=12$	F (updated)

Currently visiting: F

Visited: A, B, D, E

Unvisited: C

Node	Shortest path	Previous node
A	0	
B	2	A
D	7	B
E	8	B
F	9	D
C	12	F

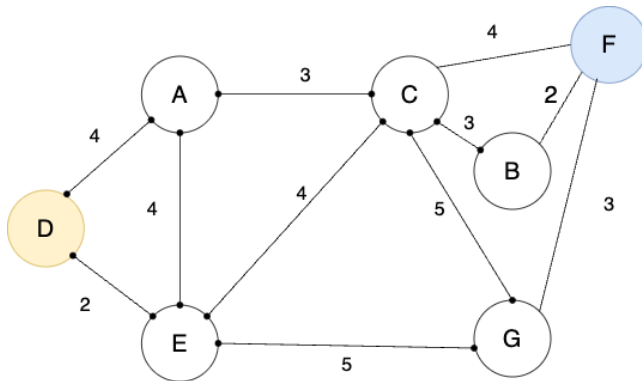
Visited: A, B, D, E, C

Unvisited: (empty)

The shortest path from A to C is 12.

Route: ABDFC.

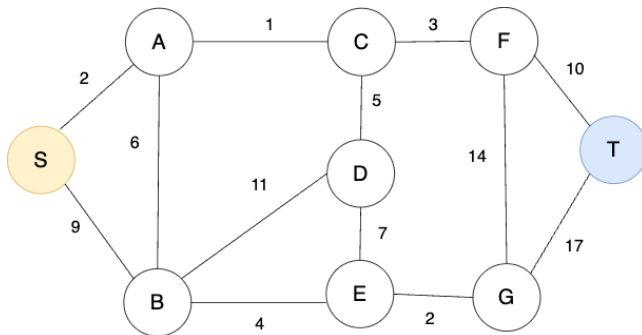
Find the shortest path from D to F



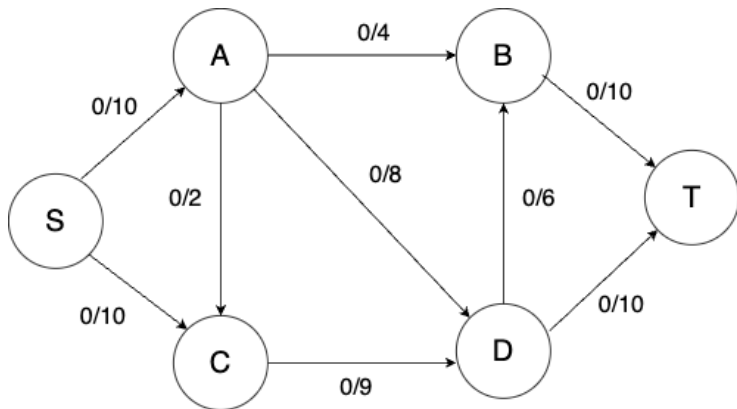
(Answer: D-E-C-F)

Application (2)

Find the shortest path from S to T



(Answer: S-A-C-F-T)



Today we learn the Ford–Fulkerson algorithm. Other methods include:

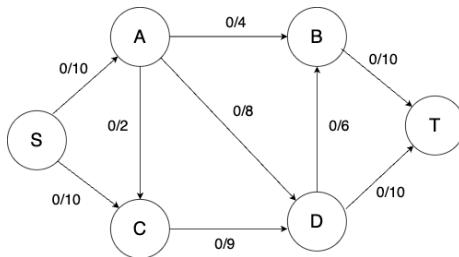
- Edmonds-Karp
- Dinitz, Push-Relabel
- with Heuristic: Goldberg-Tarjan

- 1 Find an augmenting path (non-full forward, non-empty backward)
- 2 Compute the bottleneck capacity
- 3 Augment each edge and update the total flow
- 4 Stop when we cannot find another augmenting path

Rules

- Equilibrium: inflow = outflow
- Add x to forward flow = subtract x from backward flow

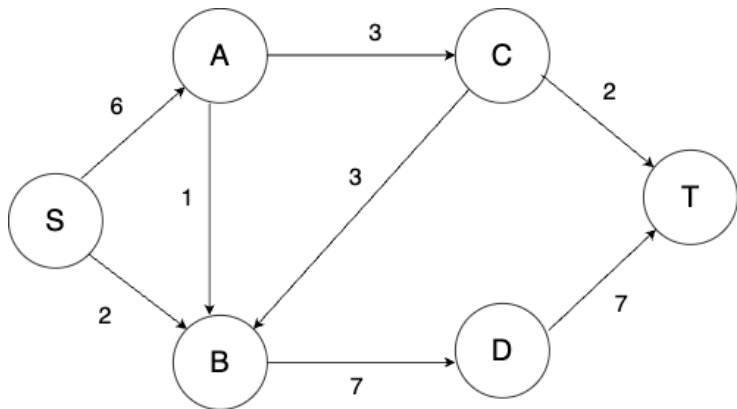
Example



- 1 iter 1: SADT. Cap = 8. Max flow = 8.
- 2 iter 2: SCDT. Cap = 2. Max flow = $8 + 2 = 10$. (since after iter 1, DT only has cap = 8)
- 3 iter 3: SCDA^{BT}. Cap = 4. Max flow = $10 + 4 = 14$.
- 4 iter 4: SA^D^{BT}. Cap = 2. Max flow = $14 + 2 = 16$.
- 5 iter 5: SC^D^{BT}. Cap = 3. Max flow = $16 + 3 = 19$.

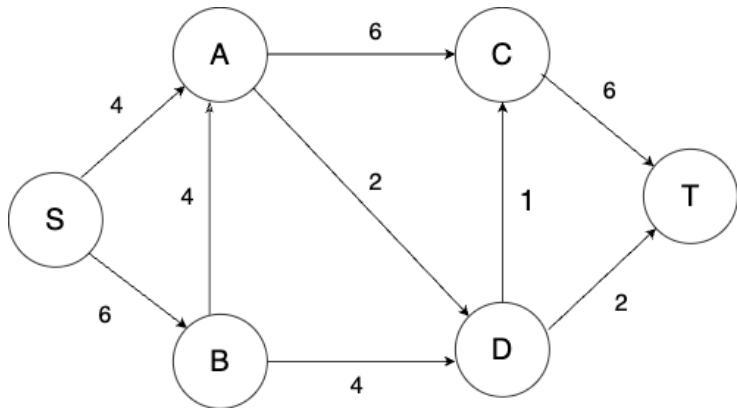
The algorithm stops because we cannot find any more augmenting paths.

Find the maximum flow of



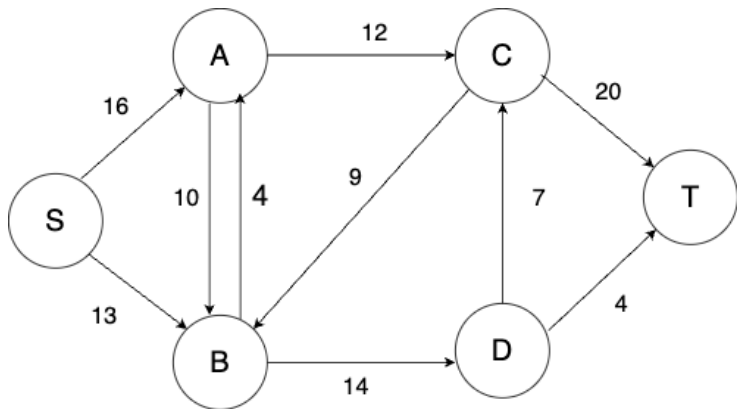
Answer: 6

Find the maximum flow of



Answer: 8

Find the maximum flow of



Answer: 23

Root Finding

One dimension. Find c such that:

$$f(c) = 0.$$

Multiple dimensions

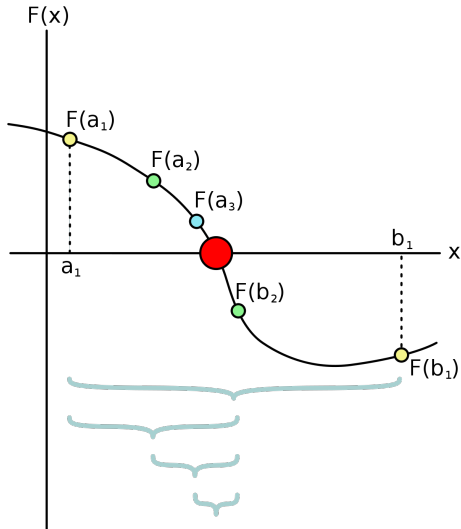
$$f_1(x_1, x_2, \dots, x_n) = 0,$$

$$f_2(x_1, x_2, \dots, x_n) = 0,$$

$$\vdots$$

$$f_n(x_1, x_2, \dots, x_n) = 0$$

We are mainly interested in 1 and 2 dimensions.



Algorithm

- 1 Choose a and b such that $f(a) \times f(b) < 0$.
- 2 Set $i = 1$.
- 3 Calculate $f(a)$.
- 4 Choose a point c where $c = a + \frac{b-a}{2}$
- 5 Calculate $f(c)$ and evaluate its sign.
 - 1 if $|f(c)| < tol$, end the program and output c .
 - 2 Else: if $f(c) \cdot f(a) > 0$, assign: $a = c$
 - 3 Else, assign $b = c$.
- 6 Repeat step 2 by setting $i = i + 1$.

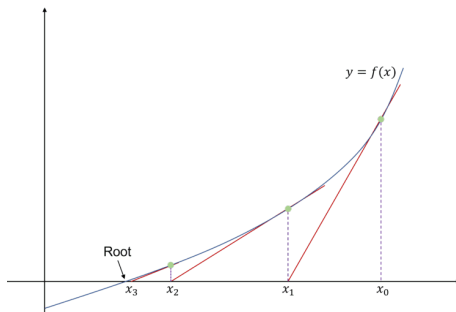
Where:

- i is the iteration
- tol is the tolerance (e.g.: $1e-6$)

Find the root of

$$f(x) = \cos(x) - x^3 + 1 = 0$$

iter	guess	error
1	2.0000	-7.4161
2	1.5000	-2.3043
3	1.2500	-0.6378
4	1.1250	0.0073
5	1.1875	-0.3006
6	1.1562	-0.1430
7	1.1406	-0.0670
8	1.1328	-0.0296
9	1.1289	-0.0111
⋮	⋮	⋮
20	1.1266	-0.0000



First-order Taylor expansion for a variable x near c has the form:

$$f(x) \approx f(c) + f'(c)(x - c)$$

Find x given $f(x) = 0$

$$x \approx c - \frac{f(c)}{f'(c)}$$

Continue updating x until x is close to c .

Algorithm:

Let n be the n^{th} iteration, the formula for the next guess.

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}. \quad (1)$$

- 1 Provide an initial guess c_0 .
- 2 Set $i = 1$.
- 3 Set $c = c_0 - \frac{f(c_0)}{f'(c_0)}$.
- 4 Calculate $\varepsilon = |c - c_0|$:
 - 1 if $\varepsilon < tol$, end the program and output c .
 - 2 Else, set $i = i + 1$, update the guess: $c_0 = c$.

Example



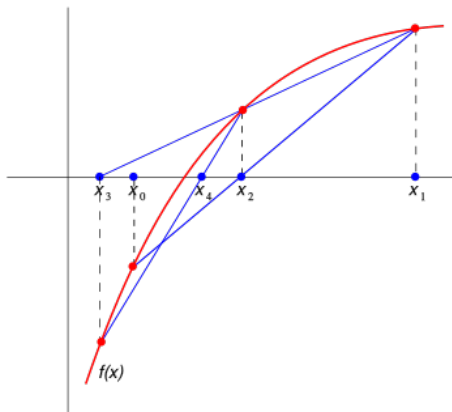
Find the root of

$$f(x) = \cos(x) - x^3 + 1 = 0$$

Compute

$$f'(x) = -\sin(x) - 3x^2$$

iter	guess	error
1	2.0056	-7.4882
2	1.4284	-1.7724
3	1.1791	-0.2577
4	1.1286	-0.0094
5	1.1266	-0.0000
6	1.1266	-0.0000



In many cases where computing the tangent line is difficult, we can use the secant line, also known as the Quasi-Newton method.

Algorithm

- 1 Provide two initial guesses c_0, c_1 .
- 2 Find $f(c_0)$ and $f(c_1)$.
- 3 Update the guess (look at Eq.(1)).

$$c = c_1 - f(c_1) \frac{c_1 - c_0}{f(c_1) - f(c_0)}.$$

- 4 Calculate $\varepsilon = |c - c_1|$:
 - 1 if $\varepsilon < tol$, end the program and output c .
 - 2 Else, set $i = i + 1$, $c_1 = c$.

In one dimension, this method is called the secant method. In multiple dimensions, it is called Broyden's method.

iter	guess	error
1	1.0393	0.3844
2	1.1361	-0.0450
3	1.1256	0.0046
4	1.1267	-0.0005
5	1.1266	0.0000
6	1.1266	-0.0000
7	1.1266	0.0000
8	1.1266	-0.0000

In multiple dimensions with

$$\mathbf{f}(\mathbf{x}) = \begin{pmatrix} f_1(x_1, \dots, x_n) \\ \vdots \\ f_n(x_1, \dots, x_n) \end{pmatrix} = \mathbf{0}$$

the algorithm is similar, but the guess is updated as follows:

$$x_{n+1} = x_n - J(x)^{-1}f(x)$$

where J is the Jacobian matrix of partial derivatives

$$J = \begin{pmatrix} f_{11} & f_{12} & \dots & f_{1n} \\ f_{21} & f_{22} & \dots & f_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ f_{n1} & f_{n2} & \dots & f_{nn} \end{pmatrix}$$

Those interested in exploring Broyden in multiple dimensions can refer [here](#).